

Taller 2 IA

Integrantes:

Ivonne Cuncanchun – 202420383

Carla González - 202411176

Samantha Puentes - 202410295

Sebastián Martínez – 202415764

Link del repositorio taller 2: <https://github.com/Carlixgonzam/Taller-2-IA>

1. Representación de estado y evaluación heurística

- Pruebe su evaluación en distintos escenarios propios, compare cualitativamente el comportamiento de sus agentes y valide sus hipótesis. Justificar qué características del estado usa (distancia a portería, cobertura de líneas de tiro, presión sobre el portador, avance del balón, etc.), qué pesos asignó y por qué.

Estados terminales dominan siempre: WIN=+1000, LOSS=-1000, DRAW=0. Para estados en progreso, la función se ramifica según posesión:

Fase	Característica	Peso
Ataque	Avance en x hacia el arco rival	3.0
	Distancia normalizada al arco rival	2.0
	Tiro libre disponible	+4.0
	Pase libre disponible (sin tiro)	+1.0
	Sin ningún carril abierto	-1.5
	Compañero adelantado al balón (c/u)	0.5
Defensa	Peligro: cercanía del balón a nuestro arco	-2.5
	Presión sobre el portador rival	-1.5
	Defensores del lado del arco propio	1.0
	Marcaje sobre rivales sin balón	-1.0
	Rival con tiro libre armado	-3.0
Ambas fases	Dispersión del equipo ósea filas distintas	0.5
	Penalización por turno	-0.5

Justificación de los pesos

- *Avance hacia el arco (3.0)*: Se asignó un peso alto porque avanzar en el territorio acerca al equipo al objetivo principal del juego, que es hacer gol. Es menor que el de un tiro libre disponible porque estar más adelante no garantiza una oportunidad inmediata de gol.
- *Distancia al arco (2.0)*: Se recompensa que el balón esté cerca del arco rival, pues aumenta la probabilidad de generar ocasiones de gol. El peso es moderado porque complementa el avance, pero por sí solo no asegura una buena jugada.
- *Tiro libre (4.0)*: Recibe el mayor peso ofensivo porque representa la situación más favorable antes de un estado terminal. Si existe una línea de tiro despejada, el agente prioriza esa oportunidad sobre seguir avanzando.
- *Pase libre (1.0)*: Peso positivo menor porque disponer de un pase libre mejora la continuidad del ataque, aunque es menos valioso que un disparo directo al arco.
- *Sin carril libre (-1.5)*: Se penaliza porque el ataque pierde opciones inmediatas y aumenta la probabilidad de perder la posesión.
- *Compañeros adelantados (0.5 c/u)*: Cada compañero adelantado representa una posible línea de pase futura. Peso pequeño para incentivar el apoyo ofensivo sin que domine sobre características más importantes.
- *Peligro cerca del arco (-2.5)*: Cuando el rival tiene el balón cerca de nuestra portería, el riesgo de recibir un gol aumenta bastante, por lo que se asigna una penalización significativa.
- *Presión sobre el portador rival (-1.5)*: Se penaliza que el rival tenga espacio y tiempo libres con el balón, porque cuanto menos presionado está, más opciones tiene de avanzar o armar una jugada peligrosa. Es menor que el peligro por cercanía porque es una amenaza indirecta.
- *Defensores del lado del arco propio (+1.0)*: Se recompensa que los jugadores de Colombia se ubiquen entre el balón y su propia portería, posición que permite interceptar un tiro o pase antes de que llegue al arco. Peso moderado: mayor que la dispersión, menor que el peligro.
- *Marcaje sobre rivales sin balón (-1.0)*: Se penaliza que un rival sin balón quede sin marca, porque es un receptor potencial de un pase peligroso. Peso menor que el peligro o el tiro libre rival porque es una prevención, no una amenaza inmediata.
- *Rival con tiro libre armado (-3.0)*: Segundo peso defensivo más alto: así como un tiro libre propio es la mejor oportunidad ofensiva, un tiro libre del rival es la peor amenaza defensiva, casi tan grave como el peligro por cercanía.
- *Dispersión (+0.5)*: Se recompensa ligeramente que los jugadores ocupen distintas filas del campo, pues incrementa las opciones de pase y evita que el equipo quede demasiado concentrado.
- *Penalización por turno (-0.5)*: Se aplica una pequeña penalización conforme avanza el partido para favorecer estrategias que resuelvan el juego rápidamente y evitar que el agente prolongue innecesariamente la partida.

Se probaron 6 escenarios con AlphaBetaAgent -d 1 donde la búsqueda exacta aísla el efecto de la heurística de cualquier ruido de la muestra:

Escenario	Resultado	Turnos
1v1_anticipation_block	DRAW	14
1v1_dribble_escape	WIN	11
2v2_attack_spread	WIN	6

2v2_defense_compact	DRAW	14
3v3_triangle_attack	DRAW	30
3v3_defense_wall	DRAW	30

Comparación cualitativa y validación de hipótesis

También se comparó contra MCTSAgent en el mismo escenario 1v1_anticipation_block: el rival arranca con un tiro libre a portería, y la única secuencia que lo evita es que Colombia se pare exactamente sobre la línea de tiro turno tras turno (el rival esquiva verticalmente, Colombia debe perseguirlo). Con AlphaBetaAgent -d 1 el resultado es **DRAW**; con MCTSAgent, en cambio, el resultado es **LOSS** de forma consistente incluso con 8000 iteraciones. Los valores intermedios calculados por la heurística son correctos en ambos casos (−3.65 justo después de bloquear el tiro, contra −1000 si no se bloquea), lo que confirma que la heurística reacciona bien; la diferencia de resultado es un problema de cómo cada algoritmo la explota; se retoma en la sección de MCTS.

Validación de pesos, usando AlphaBetaAgent -d 1 para aislar el efecto de cada peso:

- Peso tiro libre ($4.0 > 0.0$) en 1v1_dribble_escape: la secuencia de movimientos no cambia, pero el valor evaluado justo antes de disparar baja de 8.60 a 4.60 confirma que ese término aísla la señal de "carril de tiro disponible" sin afectar la construcción previa.
- Peso peligro ($2.5 > 0.0$) en 1v1_anticipation_block: el resultado no cambia (DRAW en ambos casos), porque la decisión de bloquear está dominada por evitar el LOSS terminal (−1000 fijo), no por la heurística de peligro. Confirma que los puntajes terminales dominan correctamente sobre la heurística, como se diseñó.

Conclusiones

La función de evaluación distingue con claridad ataque de defensa y responde de forma esperable a amenazas de tiro, presión e interceptación la ablación confirma que cada peso aporta la señal específica para la que fue diseñado, sin interferir con las demás. El hallazgo más importante de esta sección es que una heurística correcta no garantiza buen desempeño si el algoritmo de búsqueda no puede explotarla: con la misma función de evaluación, Alpha-Beta resuelve 1v1_anticipation_block y MCTS no. Este resultado se retoma en la sección de MCTS.

2. Rival Óptimo

Pruebas de minimax con: `python src/main.py -a MinimaxAgent -s 1v1_anticipation_block -d 1 o 2`

Pruebas alfa beta con: `python src/main.py -a AlphaBetaAgent -s 1v1_anticipation_block -d 1 o 2`

- Complejidad en tiempo y espacio (notación O) en función de la profundidad d y el factor de ramificación b (acciones joint por equipo).

Minimax tiempo: $O(b^d)$

Minimax espacio: $O(d)$

Alfa Beta tiempo:

Peor caso: $O(b^d)$

Mejor caso: $O(b^{d/2})$

Alfa Beta espacio: $O(d)$

- Describa y justifique su `evaluation_function`: ¿qué características del estado usa, qué pesos asignó y por qué?

Esto ya está mencionado en el punto 1, pero resumiendo algunos datos importantes:

Avance hacia el arco (3.0): Se asignó un peso alto porque avanzar en el territorio acerca al equipo al objetivo principal del juego, que es hacer gol. Sin embargo, este peso es menor que el de un tiro libre disponible, ya que estar más adelante no garantiza una oportunidad inmediata de gol.

Distancia al arco (2.0): Se recompensa que el balón esté cerca del arco rival, pues aumenta la probabilidad de generar ocasiones de gol. El peso es moderado porque esta característica complementa el avance, pero por sí sola no asegura una buena jugada.

Tiro libre (4.0): Recibe el mayor peso ofensivo porque representa la situación más favorable antes de un estado terminal. Si existe una línea de tiro despejada, el agente prioriza esa oportunidad sobre seguir avanzando.

Pase libre (1.0): Se asigna un peso positivo menor porque disponer de un pase libre mejora la continuidad del ataque, aunque es algo menos valioso que tener un disparo directo al arco.

Sin carril libre (-1.5): Se penaliza porque el ataque pierde opciones inmediatas y aumenta la probabilidad de perder la posesión.

Compañeros adelantados (0.5): Cada compañero adelantado representa una posible línea de pase futura. Se eligió un peso pequeño para incentivar el apoyo ofensivo sin que tal vez domine otras características más importantes.

Peligro cerca del arco (-2.5)

Cuando el rival tiene el balón cerca de nuestra portería aumenta, el riesgo de recibir un gol aumenta bastante, por lo que se asigna una penalización significativa.

Rival con tiro libre (-3)

Marcaje (-1)

Dispersión (+0.5): Se recompensa ligeramente que los jugadores ocupen distintas filas del campo, pues esto incrementa las opciones de pase y evita que el equipo quede demasiado concentrado.

Penalización por turno (-0.5): Se aplica una pequeña penalización conforme avanza el partido para favorecer estrategias que resuelvan el juego rápidamente y evitar que el agente prefiera prolongar innecesariamente la partida.

- ¿Por qué Minimax puede ser pesimista en entornos con rivales no óptimos?

Minimax asume que el rival actuará de manera óptima y elegirá la acción que minimice la utilidad MAX, es decir, en cada estado considera el peor caso posible para que jugador que está tomando la decisión. Justamente esto es lo que hace que Minimax sea un algoritmo pesimista, ya que planifica sus acciones esperando que el rival no cometa errores.

Minimax puede descartar acciones que serían muy buenas si el rival se equivocara, ya que opta por alternativas donde garantiza un buen resultado incluso frente al peor oponente posible. Esto puede hacer que el agente renuncie a oportunidades de obtener una mayor recompensa cuando el adversario no juega perfectamente.

- Compare las tasas de victoria, derrota y empate de los agentes en diversos escenarios.

Agente	Victorias	Empates	Derrotas	Tasa de victoria	Tasa de empate	Tasa de derrota
Minimax	2	2	0	50 %	50 %	0 %
Alpha-Beta	2	2	0	50 %	50 %	0 %

Tabla 1

Comparando el desempeño de los agentes Minimax y Alpha Beta en los dos escenarios (1v1_dribble_escape y 1v1_anticipation_block) y evaluando en profundidades 1 y 2. En total se realizaron 4 ejecuciones para cada agente.

Ambos agentes obtuvieron siempre los mismos resultados, como se puede observar en la tabla, y en ningún caso obtuvieron derrotas. Por lo tanto, la tasa de victoria es del 50% y la de empate del 50%.

Los resultados obtenidos eran esperados, ya que Alpha Beta implementa la misma estrategia de decisión que Minimax, solo que reduce la cantidad de nodos explorados porque hace poda. Entonces, ambos agentes tienen los mismos resultados solo que Alpha Beta requiere menor esfuerzo computacional.

- ¿Cuántos nodos adicionales poda Alpha-Beta en los mejores y peores casos?
En el mejor caso reduce de b^d a $b^{d/2}$, y en el peor caso no poda nada, es decir, explora exactamente lo mismo que Minimax.

El mejor caso ocurre cuando el algoritmo encuentra una rama cuyo valor ya es claramente peor que una alternativa conocida, puede concluir que ningún descendiente de esa rama podrá cambiar la decisión final, por lo que deja de explorarla.

NOTA: En la tabla 4 y 5 se puede ver con mejor detalle la cantidad de nodos explorados por cada algoritmo.

- Pruebe MINIMAX en escenarios con profundidad de 1 (y 2 donde el tiempo lo permita).

Escenario 1v1_anticipation_block

Algoritmo	Profundidad	Turnos	Resultado	Nodos	Tiempo (s)
Minimax	1	14	Draw	31	0.0010
Minimax	2	14	Draw	679.2	0.0163

Tabla 2

Escenario 1v1_dribble_escape

Algoritmo	Profundidad	Turnos	Resultado	Nodos	Tiempo (s)
Minimax	1	11	Victoria	26.5	0.0005
Minimax	2	11	Victoria	640.1	0.0126

Tabla 3

Se evaluó Minimax en el escenario 1v1_anticipation_block con profundidades de 1 y 2. En ambos casos el partido terminó con empate en 14 turnos máximos permitidos. Al aumentar la profundidad de búsqueda, el número promedio de nodos explorados pasó de 31 a 679.2, esto evidencia un crecimiento exponencial de costo computacional al aumentar la profundidad de búsqueda, lo cual es consistente con la complejidad exponencial de Minimax.

Se evaluó Minimax también en el escenario 1v1_dribble_escape, allí el agente logró victoria en ambas profundidades (1 y 2), finalizando el partido en 11 turnos. Al igual que en el escenario anterior, al aumentar la profundidad, aumentó significativamente el número de nodos explorados (de 26.5 a 640.1), y el tiempo promedio por decisión pasó de 0.0005s a 0.0126s, sin cambiar el resultado obtenido.

En conclusión, se observa que al aumentar la profundidad, en ambos escenarios, se incrementa significativamente el costo computacional, lo cual es consistente con lo esperado de Minimax. Sin embargo, a mayor profundidad no cambia el resultado, ya que en estos experimentos, el agente tuvo el mismo resultado para ambas profundidades en los dos escenarios evaluados.

- Verifique que AlphaBetaAgent elige exactamente las mismas acciones que MinimaxAgent explorando menos nodos

Escenario 1v1_dribble_escape

Algoritmo	Profundidad	Resultado	Turnos	Nodos	Tiempo (s)
-----------	-------------	-----------	--------	-------	------------

Minimax	1	Empate	14	31.0	0.0010
Alpha-Beta	1	Empate	14	18.5	0.0008
Minimax	2	Empate	14	679.2	0.0163
Alpha-Beta	2	Empate	14	282.9	0.0062

Tabla 4

Escenario 1v1_anticipation_block

Algoritmo	Profundidad	Resultado	Turnos	Nodos	Tiempo (s)
Minimax	1	Victoria	11	26.5	0.0005
Alpha-Beta	1	Victoria	11	17.4	0.0006
Minimax	2	Victoria	11	640.1	0.0126
Alpha-Beta	2	Victoria	11	219.4	0.0046

Tabla 5

Al comparar el comportamiento de Minimax con el de Alpha-Beta utilizando la misma función de evaluación y las mismas profundidades de búsqueda en dos escenarios distintos, los resultados indican que:

En el escenario 1v1_anticipation_block, ambos algoritmos dieron como resultado un empate para ambas profundidades, necesitando 14 turnos en ambos casos. Asimismo, en el escenario 1v1_dribble_escape, ambos agentes obtuvieron victoria para ambas profundidades, necesitando 11 turnos en ambos casos. Esto indica que Alpha-Beta tomó las mismas decisiones que Minimax en ambos experimentos.

Sin embargo, aunque los resultados fueran iguales, Alpha-Beta exploró muchos nodos menos que Minimax:

En el escenario 1v1_anticipation_block:

Con profundidad 1, Alpha Beta redujo 40.3% la cantidad de nodos explorados.

Con profundidad 2, Alpha Beta redujo 58.3% la cantidad de nodos explorados.

En el escenario 1v1_dribble_escape:

Con profundidad 1, Alpha Beta redujo 34.3% la cantidad de nodos explorados.

Con profundidad 2, Alpha Beta redujo 65.7% la cantidad de nodos explorados.

Esta disminución también redujo el tiempo promedio por decisión, especialmente en la profundidad 2, donde Alpha Beta necesitó aprox., un tercio de los nodos que exploró minimax para obtener el mismo resultado.

3. Rival Expectimax

PRUEBAS

1v1_anticipation_block

- **P=0**

```
(.venv) PS D:\IA\Taller-2-IA\AdversarialSoccer> python src/main.py -a ExpectimaxAgent -s 1v1_anticipation_block -d 2 -p 0

T2 - Adversarial Soccer - Colombia 2026
-----
Scenario   : 1v1_anticipation_block
Agent      : ExpectimaxAgent
Depth      : 2
Field      : 16x11  · 1v1  · max 14 turns
Joint branching: COL=5  RIV=6
Random rival (p): 0.0
-----
Turn 1/14 value=-3.7 · 0.04s
Turn 2/14 value=-3.7 · 0.03s
Turn 3/14 value=-3.8 · 0.03s
Turn 4/14 value=-3.8 · 0.03s
Turn 5/14 value=-3.8 · 0.03s
Turn 6/14 value=-3.9 · 0.03s
Turn 7/14 value=-3.9 · 0.03s
Turn 8/14 value=-3.9 · 0.03s
Turn 9/14 value=-4.0 · 0.03s
Turn 10/14 value=-4.0 · 0.03s
Turn 11/14 value=-4.0 · 0.04s
Turn 12/14 value=-4.1 · 0.03s
Turn 13/14 value=0.0 · 0.02s
Turn 14/14 value=0.0 · 0.00s
Match: 100% | 14/14 [00:00:00:00, 33.83turn/s, decision_s=0.00, nodes=26]
HTML graphic replay written to: D:\IA\Taller-2-IA\AdversarialSoccer\soccer_replay.html
-----
Result      : DRAW
Turns       : 14
Nodes/dec   : 679.2 · 0.0271s/dec · 1.41s total
-----

(.venv) PS D:\IA\Taller-2-IA\AdversarialSoccer>
```

- P=0.2

```
(.venv) PS D:\IA\Taller-2-IA\AdversarialSoccer> python src/main.py -a ExpectimaxAgent -s 1v1_anticipation_block -d 2 -p 0.2

T2 - Adversarial Soccer - Colombia 2026
-----
Scenario   : 1v1_anticipation_block
Agent      : ExpectimaxAgent
Depth      : 2
Field      : 16x11  · 1v1  · max 14 turns
Joint branching: COL=5  RIV=6
Random rival (p): 0.2
-----
Turn 1/14 value=-3.0 · 0.04s
Turn 2/14 value=-3.3 · 0.02s
Turn 3/14 value=-3.3 · 0.02s
Turn 4/14 value=-3.4 · 0.02s
Turn 5/14 value=-3.4 · 0.02s
Turn 6/14 value=-3.4 · 0.02s
Turn 7/14 value=-3.5 · 0.03s
Turn 8/14 value=-3.5 · 0.02s
Turn 9/14 value=-3.5 · 0.02s
Turn 10/14 value=-3.6 · 0.02s
Turn 11/14 value=-3.6 · 0.03s
Turn 12/14 value=-3.6 · 0.02s
Turn 13/14 value=0.0 · 0.01s
Turn 14/14 value=0.0 · 0.00s
Match: 100% | 14/14 [00:00:00:00, 40.88turn/s, decision_s=0.00, nodes=21]
HTML graphic replay written to: D:\IA\Taller-2-IA\AdversarialSoccer\soccer_replay.html
-----
Result      : DRAW
Turns       : 14
Nodes/dec   : 526.7 · 0.0224s/dec · 0.64s total
-----

(.venv) PS D:\IA\Taller-2-IA\AdversarialSoccer>
```

- P=0.5

```
(.venv) PS D:\IA\Taller-2-IA\AdversarialSoccer> python src/main.py -a ExpectimaxAgent -s 1v1_anticipation_block -d 2 -p 0.5

T2 - Adversarial Soccer - Colombia 2026
-----
Scenario   : 1v1_anticipation_block
Agent      : ExpectimaxAgent
Depth      : 2
Field      : 16x11  · 1v1  · max 14 turns
Joint branching: COL=5  RIV=6
Random rival (p): 0.5
-----
Turn 1/14 value=-2.1 · 0.03s
Turn 2/14 value=-2.6 · 0.02s
Turn 3/14 value=-2.7 · 0.02s
Turn 4/14 value=-2.7 · 0.02s
Turn 5/14 value=-2.7 · 0.02s
Turn 6/14 value=-2.8 · 0.02s
Turn 7/14 value=-2.8 · 0.03s
Turn 8/14 value=-2.8 · 0.03s
Turn 9/14 value=-2.9 · 0.02s
Turn 10/14 value=-2.9 · 0.03s
Turn 11/14 value=-2.9 · 0.03s
Turn 12/14 value=-3.0 · 0.03s
Turn 13/14 value=0.0 · 0.01s
Turn 14/14 value=0.0 · 0.00s
Match: 100% | 14/14 [00:00:00:00, 37.26turn/s, decision_s=0.00, nodes=21]
HTML graphic replay written to: D:\IA\Taller-2-IA\AdversarialSoccer\soccer_replay.html
-----
Result      : DRAW
Turns       : 14
Nodes/dec   : 526.7 · 0.0230s/dec · 0.69s total
-----

(.venv) PS D:\IA\Taller-2-IA\AdversarialSoccer>
```

- P: 0.8

```
PS D:\IA\Taller-2-IA\AdversarialSoccer> python src/main.py -s 1v1_anticipation_block -d 2 -p 0.8

T2 - Adversarial Soccer - Colombia 2026
Scenario : 1v1_anticipation_block
Agent : ExpectimaxAgent
Depth : 2
Field : 16x11 - 1v1 - max 14 turns
Joint branching: COL=5 RIV=6
Random rival (p): 0.8

Turn 1/14 value=-1.3 - 0.03s
Turn 2/14 value=-1.9 - 0.02s
Turn 3/14 value=-1.9 - 0.03s
Turn 4/14 value=-2.0 - 0.03s
Turn 5/14 value=-2.0 - 0.02s
Turn 6/14 value=-2.0 - 0.03s
Turn 7/14 value=-2.1 - 0.04s
Turn 8/14 value=-2.1 - 0.03s
Turn 9/14 value=-2.2 - 0.03s
Turn 10/14 value=-2.2 - 0.02s
Turn 11/14 value=-2.2 - 0.02s
Turn 12/14 value=-2.3 - 0.02s
Turn 13/14 value=0.0 - 0.01s
Turn 14/14 value=0.0 - 0.00s
Match: 100%
HTML graphic replay written to: D:\IA\Taller-2-IA\AdversarialSoccer\soccer_replay.html
Result : DRAW
Turns : 14
Nodes/dec : 526.7 - 0.0245s/dec - 0.84s total

PS D:\IA\Taller-2-IA\AdversarialSoccer>
```

- P : 1

```
PS D:\IA\Taller-2-IA\AdversarialSoccer> python src/main.py -s 1v1_anticipation_block -d 2 -p 1

T2 - Adversarial Soccer - Colombia 2026
Scenario : 1v1_anticipation_block
Agent : ExpectimaxAgent
Depth : 2
Field : 16x11 - 1v1 - max 14 turns
Joint branching: COL=5 RIV=6
Random rival (p): 1.0

Turn 1/14 value=-0.8 - 0.03s
Turn 2/14 value=-1.4 - 0.03s
Turn 3/14 value=-1.4 - 0.03s
Turn 4/14 value=-1.5 - 0.03s
Turn 5/14 value=-1.5 - 0.03s
Turn 6/14 value=-1.5 - 0.02s
Turn 7/14 value=-1.6 - 0.02s
Turn 8/14 value=-1.6 - 0.02s
Turn 9/14 value=-1.6 - 0.02s
Turn 10/14 value=-1.7 - 0.03s
Turn 11/14 value=-1.7 - 0.02s
Turn 12/14 value=-1.7 - 0.03s
Turn 13/14 value=0.0 - 0.01s
Turn 14/14 value=0.0 - 0.00s
Match: 100%
HTML graphic replay written to: D:\IA\Taller-2-IA\AdversarialSoccer\soccer_replay.html
Result : DRAW
Turns : 14
Nodes/dec : 526.7 - 0.0229s/dec - 0.79s total

PS D:\IA\Taller-2-IA\AdversarialSoccer>
```

En el escenario 1v1_anticipation_block, variar la probabilidad p del rival no modificó el desempeño de Expectimax. Independientemente del valor de p, el agente obtuvo siempre un empate en los 14 turnos disponibles. Esto indica que, para este escenario, la política calculada por Expectimax es prácticamente la misma para todas las distribuciones de probabilidad evaluadas.

2v2_attack_spread

-p = 0.5

```
PS C:\Users\ivonn\Downloads\Taller-2-IA\AdversarialSoccer> python src/main.py -a ExpectimaxAgent -s 2v2_attack_spread -d 2 -p 0.5

T2 - Adversarial Soccer - Colombia 2026
-----
Scenario : 2v2_attack_spread
Agent    : ExpectimaxAgent
Depth    : 2
Field     : 20x12 - 2v2 - max 12 turns
Joint branching: COL=26 RIV=25
Random rival (p): 0.5

Turn 1/12 value=4.2 - 7.98s
Turn 2/12 value=4.1 - 7.61s
Turn 3/12 value=4.0 - 14.15s
Turn 4/12 value=3.2 - 14.13s
Turn 5/12 value=3.3 - 4.39s
Turn 6/12 value=3.9 - 4.27s
Turn 7/12 value=3.4 - 3.94s
Turn 8/12 value=319.2 - 6.05s
Turn 9/12 value=-1.1 - 6.53s
Turn 10/12 value=-0.9 - 6.28s
Turn 11/12 value=0.0 - 1.77s
Turn 12/12 value=0.0 - 0.80s
Match: 100% | 12/12 [01:17:00:00, 6.43s/turn, decision_s=0.00, nodes=601]
HTML graphic replay written to: C:\Users\ivonn\Downloads\Taller-2-IA\AdversarialSoccer\soccer_replay.html

Result : DRAW
Turns : 12
Nodes/dec : 330137.2 - 6.4191s/dec - 77.40s total
```

-p = 1

```
PS C:\Users\ivonn\Downloads\Taller-2-IA\AdversarialSoccer> python src/main.py -a ExpectimaxAgent -s 2v2_attack_spread -d 2 -p 1

T2 - Adversarial Soccer - Colombia 2026
-----
Scenario : 2v2_attack_spread
Agent    : ExpectimaxAgent
Depth    : 2
Field     : 20x12 - 2v2 - max 12 turns
Joint branching: COL=26 RIV=25
Random rival (p): 1.0

Turn 1/12 value=4.5 - 5.82s
Turn 2/12 value=6.4 - 5.21s
Turn 3/12 value=1.1 - 6.57s
Turn 4/12 value=-0.6 - 5.53s
Turn 5/12 value=-1.1 - 4.96s
Turn 6/12 value=-1.1 - 5.16s
Turn 7/12 value=-2.0 - 4.32s
Turn 8/12 value=-18.8 - 5.34s
Turn 9/12 value=-77.2 - 4.20s
Match: 75% | 9/12 [00:47:00:15, 5.24s/turn, decision_s=4.20, nodes=273651]
HTML graphic replay written to: C:\Users\ivonn\Downloads\Taller-2-IA\AdversarialSoccer\soccer_replay.html

Result : LOSS
Turns : 9
Nodes/dec : 343019.0 - 5.2335s/dec - 47.32s total
```

- p = 1.5

```
PS C:\Users\ivonn\Downloads\Taller-2-IA\AdversarialSoccer> python src/main.py -a ExpectimaxAgent -s 2v2_attack_spread -d 2 -p 1.5

T2 - Adversarial Soccer - Colombia 2026
-----
Scenario : 2v2_attack_spread
Agent    : ExpectimaxAgent
Depth    : 2
Field     : 20x12 - 2v2 - max 12 turns
Joint branching: COL=26 RIV=25
Random rival (p): 1.5

Turn 1/12 value=6.3 - 6.20s
Turn 2/12 value=3.6 - 6.63s
Turn 3/12 value=1.7 - 4.93s
Turn 4/12 value=2.2 - 4.46s
Turn 5/12 value=2.3 - 5.34s
Turn 6/12 value=2.0 - 5.12s
Turn 7/12 value=2.5 - 3.20s
Turn 8/12 value=2.5 - 2.86s
Turn 9/12 value=2.3 - 1.98s
Turn 10/12 value=2.1 - 1.88s
Turn 11/12 value=0.0 - 0.88s
Turn 12/12 value=0.0 - 0.00s
Match: 100% | 12/12 [00:43:00:00, 3.62s/turn, decision_s=0.00, nodes=361]
HTML graphic replay written to: C:\Users\ivonn\Downloads\Taller-2-IA\AdversarialSoccer\soccer_replay.html

Result : DRAW
Turns : 12
Nodes/dec : 239801.7 - 3.6108s/dec - 43.64s total
```

En el escenario 2v2_attack_spread, la elección del parámetro p sí afectó el comportamiento del agente. Con p=0.5 y p=1.5 el resultado fue un empate, mientras que con p=1.0 el agente terminó perdiendo el partido. Esto muestra que la política obtenida por Expectimax depende de la distribución probabilística asumida para el rival y que un mismo escenario puede favorecer o perjudicar determinadas elecciones de p. En

consecuencia, la selección de este parámetro requiere ajustarse al comportamiento esperado del oponente.

EVALUATION FUNCTION

La función de evaluación asigna un puntaje a cada estado del juego según qué tan favorable sea para Colombia. Para esto tuve en cuenta aspectos como la posesión del balón, la distancia al arco rival, la presión del rival y las oportunidades de ataque. La idea fue dar un mayor puntaje a las posiciones donde Colombia tiene más posibilidades de anotar y uno menor cuando el rival tiene ventaja. Además, utilicé pesos balanceados para que ninguna característica tuviera más importancia que las demás.

COMPLEJIDAD

Complejidad temporal: $O(bd)$

En cada nivel del árbol se evalúan todas las acciones posibles de Colombia y del rival para calcular el valor esperado, por lo que el tiempo aumenta con la profundidad de búsqueda.

Complejidad espacial: $O(d)$

La memoria utilizada depende principalmente de la profundidad de la recursión durante la búsqueda.

Comparación entre minimax y Alpha beta y Expectimax

Expectimax se diferencia de Minimax porque no asume que el rival siempre juega de la mejor manera, sino que considera la posibilidad de que cometa errores. Por otro lado, Alpha-Beta es más eficiente porque reduce la cantidad de nodos explorados mediante poda, mientras que Expectimax debe evaluar todas las acciones para calcular el valor esperado. En las pruebas realizadas el resultado fue empate y, al aumentar el valor de p , el agente tomó decisiones menos conservadoras.

CONCLUSIONES

En conclusión, Expectimax funcionó bien y permitió simular un rival que no siempre toma la mejor decisión. También pude ver que el valor de p cambia el comportamiento del agente: entre más alto es, más tiene en cuenta que el rival puede equivocarse. Me pareció un algoritmo útil porque representa mejor situaciones reales que Minimax cuando el rival no juega de forma perfecta.

5. MCTS

- Complejidad en tiempo y espacio (notación O) en función del número de iteraciones N y la ramificación del estado

Complejidad tiempo:

$O(N*b)$

Si se ejecutan N iteraciones, el algoritmo se repite las 4 fases N veces. En cuanto al factor de ramificación b, este influye en el costo de seleccionar la mejor acción mediante UCT, ya que para cada nodo deben evaluarse sus hijos y en el peor caso se deben revisar hasta b acciones posibles.

Complejidad Espacial:

$O(N)$

Durante la ejecución MCTS construye de manera incremental el árbol de búsqueda. En cada iteración se agrega máximo un nuevo nodo entonces después de N iteraciones el árbol contiene aprox. N nodos.

- ¿En qué escenarios supera MCTS o no a Minimax y Expectimax? Justifique con experimentos

MCTS pierde en 1v1_anticipation_block: la secuencia ganadora requiere acertar 1 acción de 5 posibles en cada turno, y el rollout aleatorio rara vez encuentra esa secuencia completa — confirmado con 8000 iteraciones sin mejora (LOSS consistente). Minimax/Alpha-Beta con solo profundidad 1 lo resuelven porque exploran todas las respuestas del rival, no solo una muestra al azar.

MCTS gana en escenarios grandes donde Minimax/Alpha-Beta se vuelven imprácticos: en 3v3_triangle_attack, calcular una sola decisión de Minimax a profundidad 2 tardó más de 35 segundos sin terminar (lo interrumpí), mientras que MCTS con apenas 150-500 iteraciones completa un partido entero (26-30 turnos) en 17-82 segundos totales. Es decir, Minimax d=2 no puede ni tomar una sola decisión en el tiempo que MCTS necesita para jugar el partido completo.

Ahora la comparación con expectimax:

Algoritmo	Escenario	Parámetro	Resultado	Turnos	Tiempo total	Tiempo/decisión
Expectimax	2v2_attack_spread	d = 2	DRAW	12	90.27 s	7.4321 s
MCTS	2v2_attack_spread	150 iteraciones	WIN	5	0.81 s	0.1057 s
MCTS	2v2_attack_spread	500 iteraciones	WIN	10	9.16 s	0.7352 s

En el escenario 2v2_attack_spread, Expectimax con profundidad 2 logró completar el partido, pero requirió 90.27 segundos de tiempo total y un promedio de 7.43 segundos por decisión, explorando aproximadamente 330137 nodos por decisión.

Por el contrario, MCTS obtuvo un mejor resultado (victoria) utilizando únicamente simulaciones:

- Con 150 iteraciones ganó el partido en 5 turnos, requiriendo únicamente 0.81 segundos en total.
- Con 500 iteraciones también obtuvo una victoria, finalizando el partido en 10 turnos y empleando 9.16 segundos.

Estos experimentos muestran que, aunque Expectimax realiza una búsqueda mucho más exhaustiva, su costo computacional aumenta rápidamente con el tamaño del árbol de búsqueda. En cambio, MCTS produce decisiones de buena calidad mediante simulaciones, logrando tiempos considerablemente más cortos y un mejor desempeño práctico en este escenario.

En conclusión, en escenarios pequeños (como 1v1_dribble_escape) Minimax, Alpha Beta y Expectimax encuentran soluciones óptimas en tiempos muy cortos. Sin embargo, cuando aumenta el factor de ramificación, Minimax y Expectimax presentan un crecimiento exponencial del costo computacional. En 2v2_attack_spread, Expectimax necesitó más de 90 s para completar un partido, mientras que MCTS obtuvo una victoria en menos de 10 s incluso con 500 iteraciones. En escenarios aún más grandes, como 3v3_triangle_attack, Minimax y Expectimax con profundidad 2 dejaron de ser prácticos, mientras que MCTS continuó resolviendo los partidos en tiempos razonables.

- Relación entre número de iteraciones y calidad de juego / tiempo por turno para distintos escenarios

Iteraciones vs calidad de juego / tiempo por turno

Escenario	Iteraciones	Resultado	Tiempo/turno
1v1_dribble_escape	10	WIN	0.0011 s
1v1_dribble_escape	50	WIN	0.0035 s
1v1_dribble_escape	150	WIN	0.0083 s
1v1_dribble_escape	500	WIN	0.0225 s
1v1_dribble_escape	1500	WIN	0.0334 s
2v2_attack_spread	10	DRAW	0.0054 s
2v2_attack_spread	50	WIN	0.0277 s
2v2_attack_spread	150	DRAW	0.0754 s
2v2_attack_spread	500	DRAW	0.2477 s
2v2_attack_spread	1500	WIN	0.6055 s

Tabla 6

El tiempo promedio por turno crece aproximadamente lineal con las iteraciones de MCTS (esperable: cada iteración es $O(1)$ de trabajo adicional), ya que cada iteración ejecuta una nueva fase.

En cuanto a la calidad, en cambio, no mejora monótonamente al incrementar el número de iteraciones. En 1v1_dribble_escape, el agente tuvo una victoria con todas las

cantidades de iteraciones evaluadas, lo que indica que se trata de un escenario relativamente sencillo y que pocas simulaciones son suficientes para encontrar una buena decisión. En 2v2_attack_spread los resultados alternan entre WIN y DRAW según las iteraciones.

Esto ocurre porque MCTS utiliza simulaciones aleatorias para estimar el valor de las acciones; con un número limitado de iteraciones aún existe variabilidad en dichas estimaciones. Al aumentar las iteraciones, el algoritmo dispone de más información y sus estimaciones son más estables, aunque esto no implica necesariamente obtener un mejor resultado en todas las ejecuciones.

6. Reflexión Sobre el Uso de IA

El uso que le dimos a la inteligencia artificial para la elaboración de este taller fue enriquecedor en el sentido que nos permitió encontrar errores e inconsistencias en el código que implementamos sin saber previamente por qué ocurrían. Esto permite un aprendizaje casi efectivo diciéndole a la IA la causa de esos errores durante su creación para que de este modo nos retáramos nuevamente a volverlo a intentar y como hay veces que usando nuestra lógica humana falla en ciertos casos.

Finalmente, pedirle a la IA explicaciones graficas previas para saber cómo implementar el algoritmo antes de fue una gran ayuda, ya que no se le pedía inmediatamente una solución rápida, sino darle un rol de tutor donde explica el trasfondo del algoritmo y unas posibles pistas para la implementación.

Cada uso de prompts y de outputs se encuentran en los respectivos archivos .py las cuales también hay documentación y en algunos casos, el proceso de elaboración de los algoritmos.